Operating Systems Course
Project Handout

CPU Scheduling Comparison Projects

Student-facing formal handout for five approved comparison-project variants

This handout applies to the Operating Systems project. Each team is assigned one comparison project and must implement both algorithms, run them on the same workloads, and produce a technically sound comparison with clear metrics, Gantt charts, validation, and written analysis.

1. Purpose of this Handout

This document defines the five approved comparison projects for the Operating Systems course project.

In every project, the team must implement two scheduling algorithms, execute them on the same workload, and explain the differences in behavior, performance, fairness, and scheduling trade-offs.

The goal is not only to build a simulator, but also to demonstrate understanding of CPU scheduling concepts through accurate comparison and technical interpretation.

2. Common Requirements for All Five Projects

Every project must include a clear interface, separate Gantt charts for both algorithms, user-defined process input at runtime, input validation, per-process Waiting Time (WT), Turnaround Time (TAT), and Response Time (RT), plus average WT, average TAT, and average RT.

All comparisons must use the same workload for both algorithms. A comparison is not valid if the input dataset differs between the two runs.

At minimum, the simulator must accept Process ID, Arrival Time, and Burst Time. Priority-based projects must also accept Priority Value. Round Robin projects must also accept Time Quantum.

The simulator must safely reject invalid data such as negative arrival times, zero or negative burst times, duplicate process IDs, invalid priority values, invalid quantum values, missing required fields, and non-numeric input in numeric fields.

3. Approved Projects

Code	Project	Algorithms Compared	Main Focus	Extra Input
C1	Priority vs SRTF	Priority + SRTF	Policy vs shortest remaining time	Priority
C2	Round Robin vs SRTF	RR + SRTF	Fairness vs efficiency	Quantum
C3	Round Robin vs Priority	RR + Priority	Fairness vs urgency	Quantum + Priority
C4	SJF vs Priority	SJF + Priority	Shortest job vs urgency	Priority
C5	Round Robin vs SJF	RR + SJF	Time slicing vs shortest job	Quantum

Priority vs SRTF Comparison Project

Project at a glance: Algorithms compared: **Priority Scheduling** and **Shortest Remaining Time First (SRTF)**. Main focus: policy-driven service versus shortest-remaining-time efficiency. **Special input**: Priority value for each process.

Project Objective

In this project, your team will implement and compare Priority Scheduling and Shortest Remaining Time First (SRTF).

The purpose is to study the difference between a scheduler that selects processes according to priority values and a scheduler that selects according to the shortest remaining burst time.

Your comparison should reveal how scheduling policy affects fairness, preemption, starvation risk, and the final performance metrics.

Required Functionality

- Accept a dynamic number of processes and all required process data at runtime.
- Validate all input before simulation begins.
- Simulate Priority Scheduling correctly, with a clearly documented priority rule and tie-breaking rule.
- Simulate SRTF correctly, including immediate preemption when a shorter remaining job arrives.
- Display separate Gantt charts and separate per-process metrics tables for both algorithms.
- Calculate WT, TAT, RT, average WT, average TAT, and average RT.

Required Comparison Focus

- How Priority Scheduling behaves when a short job has low priority.
- How SRTF behaves when a longer but high-priority process competes with shorter jobs.
- Fairness versus efficiency.
- Starvation risk and policy-driven service.

Required Interface Sections

- Input Panel
- Process Table
- Gantt Chart for Priority
- Gantt Chart for SRTF
- Results Table for Priority
- Results Table for SRTF
- Comparison Summary Section
- Final Conclusion Area

Required Test Scenarios

Scenario A: Basic mixed workload

- A normal workload with multiple processes, different arrival times, and different burst times.

Scenario B: Conflict between priority and burst time

- Include a high-priority long process and a low-priority short process so the two algorithms behave differently.

Scenario C: Starvation-sensitive case

- Prepare a workload where one process may wait much longer depending on the selected policy.

Scenario D: Validation case

- Include at least one invalid input example and show how the simulator handles it safely.

Required Analysis Questions

- Which algorithm produced the lower average waiting time?
- Which algorithm produced the lower average response time?
- Did priority values improve treatment of urgent processes?
- Did SRTF favor short jobs more aggressively?
- Which algorithm would you recommend for the tested workload, and why?

Required Conclusion

- State which algorithm performed better on the selected datasets.
- State which metrics were better under each algorithm.
- Identify the main trade-off observed.
- State which algorithm appeared fairer in practice.

Round Robin vs SRTF Comparison Project

Project at a glance: Algorithms compared: **Round Robin** and **Shortest Remaining Time First (SRTF)**. Main focus: time-slicing fairness versus shortest-remaining-time efficiency. **Special input:** Time Quantum.

Project Objective

In this project, your team will implement and compare Round Robin and SRTF.

The purpose is to compare a fairness-oriented time-slicing scheduler with an efficiency-oriented shortest-remaining-time scheduler.

Your analysis should reveal how response time, waiting time, fairness, preemption, and quantum choice affect system behavior.

Required Functionality

- Accept a dynamic number of processes and all required process data at runtime.
- Validate all input and reject invalid quantum values safely.
- Simulate Round Robin correctly with a user-defined time quantum and a correct ready queue.
- Simulate SRTF correctly with immediate preemption when required.
- Display separate Gantt charts and separate metrics tables for both algorithms.
- Calculate WT, TAT, RT, average WT, average TAT, and average RT.

Required Comparison Focus

- Fairness versus efficiency.
- Effect of time slicing versus shortest-job preference.
- Effect on first response time.
- Effect of quantum size on Round Robin behavior.
- Whether SRTF gives a strong advantage to short jobs.

Required Interface Sections

- Input Panel
- Quantum Input Field
- Ready Queue View for Round Robin
- Gantt Chart for Round Robin
- Gantt Chart for SRTF
- Results Table for Round Robin
- Results Table for SRTF
- Comparison Summary Panel
- Final Conclusion Area

Required Test Scenarios

Scenario A: Basic mixed workload

- Use a normal workload with several processes.

Scenario B: Quantum sensitivity case

- Run a workload with a clearly stated quantum that meaningfully affects results.

Scenario C: Short-job-heavy case

- Use a workload with many short jobs so SRTF behavior becomes very visible.

Scenario D: Interactive-style fairness case

- Use a workload designed to show whether Round Robin gives faster first response to multiple processes.

Scenario E: Validation case

- Include at least one invalid input example and show the validation behavior.

Required Analysis Questions

- Which algorithm gave better average waiting time?
- Which algorithm gave better response time?
- Did Round Robin appear fairer across all processes?
- Did SRTF complete short jobs faster?
- How did the selected quantum affect the Round Robin results?
- Which algorithm would you recommend for the tested workload, and why?

Required Conclusion

- State which algorithm performed better on each major metric.
- State whether Round Robin appeared fairer.
- State whether SRTF appeared more efficient.
- Explain the observed effect of the selected quantum.

Round Robin vs Priority Comparison Project

Project at a glance: Algorithms compared: **Round Robin** and **Priority Scheduling**. Main focus: fairness versus urgency and service differentiation. **Special input**: Time Quantum and Priority value.

Project Objective

In this project, your team will implement and compare Round Robin and Priority Scheduling.

The purpose is to compare a scheduler that shares CPU time across ready processes with a scheduler that favors more important processes according to priority.

Your analysis should reveal how urgency, fairness, starvation risk, and policy-driven service affect execution order and metrics.

Required Functionality

- Accept a dynamic number of processes and all required process data at runtime.
- Validate all input, including quantum and priority values.
- Simulate Round Robin correctly with proper ready-queue rotation and arrival handling.
- Simulate Priority Scheduling correctly with a clearly documented priority rule and tie-breaking rule.
- Display separate Gantt charts and separate results tables for both algorithms.
- Calculate WT, TAT, RT, average WT, average TAT, and average RT.

Required Comparison Focus

- Fairness versus urgency.
- How priority changes execution order.
- Whether urgent processes benefit significantly.
- Whether low-priority processes may suffer starvation.
- Whether Round Robin gives more balanced service.

Required Interface Sections

- Input Panel
- Quantum Input Field
- Priority Input Field
- Process Table
- Gantt Chart for Round Robin
- Gantt Chart for Priority
- Results Table for Round Robin
- Results Table for Priority
- Comparison Summary Section
- Final Conclusion Area

Required Test Scenarios

Scenario A: Basic mixed workload

- Use a normal workload with several processes.

Scenario B: Urgency case

- Include one or more processes with clearly higher priority so the policy difference is visible.

Scenario C: Fairness case

- Prepare a workload that shows whether Round Robin distributes service more evenly.

Scenario D: Possible starvation case

- Prepare a workload in which low-priority processes may wait much longer.

Scenario E: Validation case

- Include at least one invalid input example and show how the simulator handles it.

Required Analysis Questions

- Which algorithm gave better average waiting time?
- Which algorithm gave better response time?
- Did higher-priority processes gain significant advantage?
- Did Round Robin appear more balanced across all processes?
- Was starvation observed or likely in Priority Scheduling?
- Which algorithm would you recommend for the tested workload, and why?

Required Conclusion

- State which algorithm performed better on the selected datasets.
- State whether priority-based service improved urgent-task treatment.
- State whether Round Robin improved fairness.
- State whether starvation risk appeared.

SJF vs Priority Comparison Project

Project at a glance: Algorithms compared: **Shortest Job First (SJF)** and **Priority Scheduling**. Main focus: shortest available job versus urgency or importance. **Special input**: Priority value.

Project Objective

In this project, your team will implement and compare Shortest Job First (SJF) and Priority Scheduling.

The purpose is to compare a scheduler that prefers the shortest available burst time with a scheduler that prefers the highest-priority process according to a defined rule.

Your team is expected to study how job length versus urgency affects execution order, average metrics, fairness, and starvation risk.

Required Functionality

- Accept a dynamic number of processes and all required process data at runtime.
- Validate all input safely before simulation begins.
- Implement SJF correctly; unless otherwise instructed, SJF should be non-preemptive.
- Implement Priority Scheduling correctly, with a clearly stated priority rule and documented tie handling.
- Display separate Gantt charts and separate metrics tables for both algorithms.
- Calculate WT, TAT, RT, average WT, average TAT, and average RT.

Required Comparison Focus

- How SJF behaves when a short job has low priority.
- How Priority behaves when a long job has very high priority.
- Whether SJF improves average waiting or turnaround time.
- Whether Priority improves service for urgent processes.
- Whether either algorithm causes unfair delay.

Required Interface Sections

- Input Panel
- Process Table
- Priority Input Area
- Gantt Chart for SJF
- Gantt Chart for Priority
- Results Table for SJF
- Results Table for Priority
- Comparison Summary Section
- Final Conclusion Area

Required Test Scenarios

Scenario A: Basic mixed workload

- Use a normal workload with different arrival times and burst times.

Scenario B: Conflict between burst time and priority

- Include a short-burst low-priority process and a long-burst high-priority process to reveal a meaningful difference.

Scenario C: Fairness or starvation-sensitive case

- Prepare a workload where one process may wait much longer under one of the algorithms.

Scenario D: Validation case

- Include at least one invalid input example and show the validation behavior.

Required Analysis Questions

- Which algorithm gave lower average waiting time?
- Which algorithm gave lower average turnaround time?
- Did SJF favor short jobs more strongly?
- Did Priority Scheduling favor urgent processes more strongly?
- Was any starvation or unfair delay observed?
- Which algorithm would you recommend for the tested workload, and why?

Required Conclusion

- State which algorithm performed better on the selected datasets.
- State which metric each algorithm handled better.
- Explain the trade-off between efficiency and urgency.
- State which algorithm appeared fairer in practice.

Round Robin vs SJF Comparison Project

Project at a glance: Algorithms compared: **Round Robin** and **Shortest Job First (SJF)**. Main focus: time slicing and balance versus shortest-job efficiency. **Special input**: Time Quantum.

Project Objective

In this project, your team will implement and compare Round Robin and Shortest Job First (SJF).

The purpose is to compare a scheduler that shares CPU time in a rotating, time-sliced manner with a scheduler that selects the shortest available job first.

Your team is expected to study differences in response time, waiting time, turnaround time, execution fairness, and workload suitability.

Required Functionality

- Accept a dynamic number of processes and all required process data at runtime.
- Validate all input and reject invalid quantum values safely.
- Implement Round Robin correctly with proper queue rotation, arrival handling, and remaining-burst tracking.
- Implement SJF correctly; unless otherwise instructed, SJF should be non-preemptive.
- Display separate Gantt charts and separate metrics tables for both algorithms.
- Calculate WT, TAT, RT, average WT, average TAT, and average RT.

Required Comparison Focus

- Fairness versus efficiency.
- How Round Robin distributes CPU time.
- How SJF favors short jobs.
- Effect of the selected quantum on Round Robin behavior.
- Effect on first response time and long-process treatment.

Required Interface Sections

- Input Panel
- Quantum Input Field
- Process Table
- Ready Queue View for Round Robin
- Gantt Chart for Round Robin
- Gantt Chart for SJF
- Results Table for Round Robin
- Results Table for SJF
- Comparison Summary Panel
- Final Conclusion Area

Required Test Scenarios

Scenario A: Basic mixed workload

- Use a normal workload with several processes.

Scenario B: Short-job-heavy case

- Use a workload with several short jobs so SJF behavior becomes very visible.

Scenario C: Fairness case

- Prepare a workload that shows how Round Robin distributes service across multiple processes.

Scenario D: Long-job sensitivity case

- Use a workload where one long process competes with shorter processes and the outcome differs between the algorithms.

Scenario E: Validation case

- Include at least one invalid input example and show the validation behavior.

Required Analysis Questions

- Which algorithm gave lower average waiting time?
- Which algorithm gave lower average response time?
- Did Round Robin appear fairer across all processes?
- Did SJF complete short jobs more efficiently?
- How did the chosen quantum affect Round Robin behavior?
- Which algorithm would you recommend for the tested workload, and why?

Required Conclusion

- State which algorithm performed better on each major metric.
- State whether Round Robin appeared more balanced.
- State whether SJF appeared more efficient.
- Explain the observed effect of the selected quantum.